VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Database system (CO2013)

Assignment 2

Hospital Database Implementation

Supervisor: Prof. Phan Trong Nhan

Students: Do Thanh Trung – 2252853
Tran Gia Linh – 2252434
Huynh Nguyen Ngoc Anh – 2252022
Nguyen Trinh Tien Dat – 2252147
Huynh Thanh Duy – 2252114

HO CHI MINH CITY, NOVEMBER 2024

Full name	ID	Task	Contribution
Huynh Nguyen Ngoc Anh	2252022	Back end development + Create user	20%
Tran Gia Linh	2252434	Script SQL for tables and data	20%
Do Thanh Trung	2252853	Indexing + Security	20%
Huynh Thanh Duy	2252114	Front end development	20%
Nguyen Trinh Tien Dat	2252147	Figma + Part 2	20%

Contents

1 Physical database design

The physical database design section outlines the practical implementation of the database schema, its constraints, and supporting components like triggers.

1.1 EERD and Relational Mapping

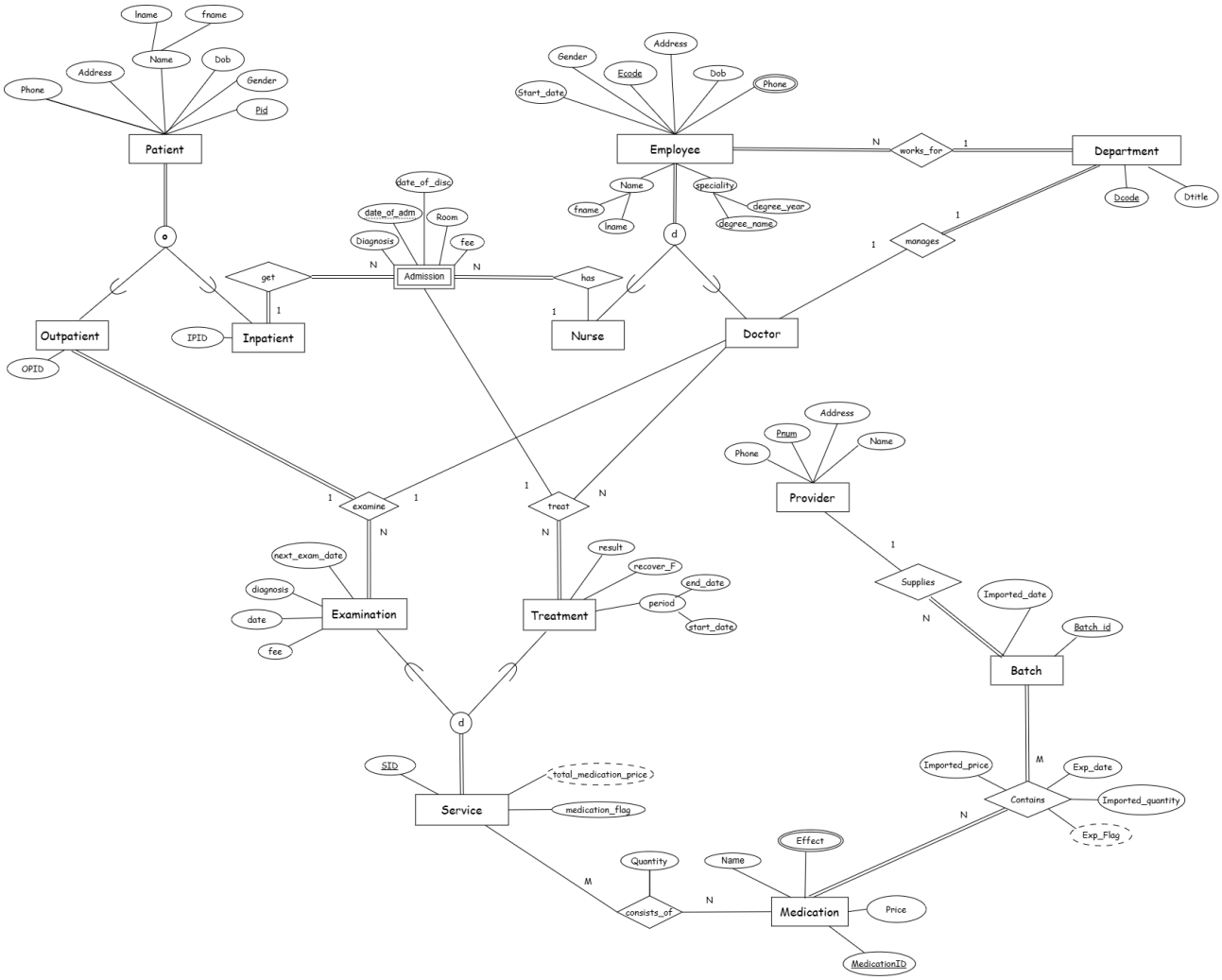


Figure 1.1: EERD of hospital database

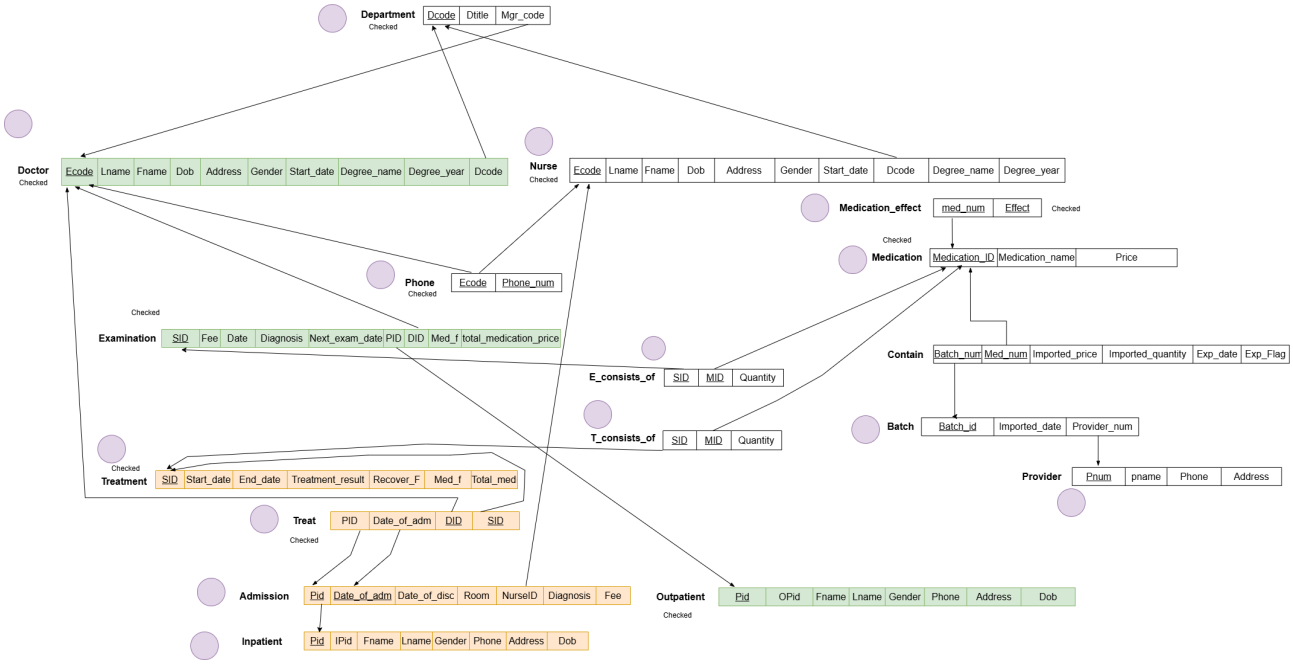


Figure 1.2: Relational mapping of hospital database

1.2 Physical design

Department				
Column	Data type	Data length	Constraints	Explanation
dcode	smallint	2 bytes	- Primary key - Auto-increment	Each dcode can be a different integer value, Also the range of small int is -32768 to +32767, so it is appropriate since the number of department is not likely to exceed 32767.
dtitle	varchar		NOT NULL	The name of the department in a hospital can have any length, however it is unlikely that it exceed 100 characters. Also it must not be NULL.
mgr_code	integer	8 bytes	- Foreign key, references Doctor(ecode)	A department may not have any dean so it can be NULL.

Foreign key behavior:

mgr_code:

- On update cascade: Update as ecode of current dean is updated.
- On delete set null: If current dean no longer work for hospital, the dean position of the department will be empty.

doctor_phone/ nurse_phone				
Column	Data type	Data length	Constraints	Explanation
ecode	integer	4 bytes	- NOT NULL, FK	Each ecode is a different integer value. It is shared between Nurse and Doctor since both "ecode" attributes are inherited from Employee(ecode) in schema.
phone_num	char(10)	10 bytes	- NOT NULL - Must have exactly 10 digits	Assume that the hospital is in Vietnam. A phone number has exactly 10 digits.
Primary key	(ecode, phone_num)			

Foreign key behavior:

ecode:

- On update cascade: update as ecode of employee is updated.
- On delete cascade: is deleted as the employee information is deleted.

Doctor/Nurse				
Column	Data type	Data length	Constraints	Explanation
ecode	integer	4 bytes	NOT NULL	Each ecode is a different integer value. It is shared between Nurse and Doctor since both "ecode" attributes are inherited from Employee(ecode) in schema.
lname	varchar(20)	At most 20 bytes	NOT NULL	Employee 's personal information is important. Therefore, not null.
fname	varchar(20)	At most 20 bytes	- Auto-generated - Outpatient: OPXXXXXXXXX - Inpatient: IPXXXXXXXXX, X is digit.	Employee 's personal information is important. Therefore, not null.
dob	date	4 bytes	NOT NULL	Employee 's personal information is important. Therefore, not null.
address	varchar		NOT NULL	Employee 's personal information is important. Therefore, not null.
gender	char(1)	1 bytes	- NOT NULL - Is either 'F' or 'M'	Employee 's personal information is important. Therefore, not null.
start_date	date	4 bytes	NOT NULL	If an employee officially works for the hospital, start_date must be provided.
degree_name	varchar(100)	At most 100 bytes	NOT NULL	Doctor or nurse must have a degree to work at a hospital.
degree_year	smallint	2 bytes	- NOT NULL - Smaller than current year, larger than 1900	If degree_year is larger than current year, the degree is not valid.
dcode	smallint	2 bytes	- NOT NULL - Foregin key, reference department(dcode)'	An employee must work for a specific department.

Foreign key behavior:

dcode:

- On update cascade: Update as dcode of current department is updated.
- On delele restrict: If there is at least one employee working for a department, that department cannot be deleted.

Provider				
Column	Data type	Data length	Constraints	Explanation
pnum	integer	4 bytes	Primary key Auto-increment	Automatically assigns a unique ID to each provider. Ensures unique identification.
pname	varchar(50)	At most 50 bytes	NOT NULL	Provider names cannot be null as it is necessary for identification.
phone	char(10)	10 bytes	Must be exactly 10 digits if not null.	Assumes phone numbers in Vietnam have 10 digits.
address	varchar		NOT NULL	Origin of the medication is important.

Batch				
Column	Data type	Data length	Constraints	Explanation
batch_id	integer	4 bytes	Primary key Auto-increment	Unique identifier for each batch of medication.
imported_date	date	4 bytes	NOT NULL	Imported date must not be null in order to trace back the medication origin if needed.
provider_num	integer	4 bytes	NOT NULL Foreign key, referencing to Provider(pnum)	

Foreign key behavior:

provider_num:

- On update cascade: Update as pnum of current provider is updated.
- On delete restrict: If there is at least a batch provided by a particular provider, that provider must not be deleted.

Foreign key behavior:

batch_num:

- On update cascade: update as batch_id is updated.
- On delete restrict: Batch number of medication that is still recorded in medication storage of the hospital cannot be deleted.

med_num:

- On update cascade: same logic as above.

Inpatient / Outpatient				
Column	Data type	Data length	Constraints	Explanation
pid	integer	4 bytes	- Primary key. - Auto-increment, shared sequence between inpatient and outpatient.	Shared sequence between inpatient and outpatient because one person who is both inpatient and outpatient share the same pid in both relations.
ipid	char(11)	11 bytes	- Auto-generated - Outpatient: OPXXXXXXXXX - Inpatient: IPXXXXXXXXX, X is digit.	Because it contains both number and character, char type is chosen. Also, the digits part will match the pid of the patient with additional padding if needed.
lname	varchar(20)	At most 20 bytes		It is likely that data is incorrect if last name's length is more than 20
fname	varchar(20)	At most 20 bytes		It is likely that data is incorrect if first name's length is more than 20
dob	date	4 bytes	NOT NULL	Age of a patient is important in regards of performing health service, so it should not be null.
address	varchar			
phone	char(11)	11 bytes	- If not NULL, must be 10 digits.	Assume that the hospital is in Vietnam where phone number has exactly 10 digits.
gender	char(1)	1 byte	- NOT NULL - Is either 'F' or 'M'	Gender of a patient is taken into consideration when performing health service.

Medication				
medication_id	Data type	Data length	Constraints	Explanation
batch_id	integer	4 bytes	Primary key Auto-increment	Unique identifier for each medication.
name	varchar(100)	At most 100 bytes	NOT NULL	Stores the medication name; duplicates are allowed for different sizes or types.
price	integer	4 bytes	NOT NULL	Since price is for each unit, integer is long enough.

Contain				
Column	Data type	Data length	Constraints	Explanation
batch_num	integer	4 bytes	NOT NULL Foreign Key	
med_num	integer	4 bytes	NOT NULL Foreign Key	
imported_price	integer	4 bytes	NOT NULL	Integer is large enough.
imported_quantity	integer	4 bytes	NOT NULL	Must not be null to keep track of the medication storage
exp_date	date	4 bytes	NOT NULL	Must not be null since expiration date of medication is sensitive.
exp_flag	boolean	1 byte	NOT NULL, Default is False	Derived from exp_date
Primary key	(batch_num, med_num)			

– On delete restrict: same logic as above.

Medication Effect				
Column	Data type	Data length	Constraints	Explanation
med_num	integer	4 bytes	NOT NULL Foreign Key	
effect	varchar(100)	At most 100 bytes	NOT NULL	
Primary key	(med_num, effect)			

Foregin key behavior:

med_num:

- On update cascade: update as med_id of the medication in the refereneced table is updated.
- On delete cascade: If the medication is deleted, its effects information is also deleted.

Medication Effect				
Column	Data type	Data length	Constraints	Explanation
med_num	integer	4 bytes	NOT NULL Foreign Key	
effect	varchar(100)	At most 100 bytes	NOT NULL	
Primary key	(med_num, effect)			

Admission				
Column	Data type	Data length	Constraints	Explanation
pid	integer	4 bytes	NOT NULL, Foreign Key	Unique identifier for each treatment.
date_of_adm	date	4 bytes	NOT NULL	Whenever patient is admitted, the date must be recorded.
date_of_disc	date	4 bytes	Either NULL or date_of_disc >= date_o_adm	Can be NULL since the patient has not been discharged.
room	varchar			
nurse_id	integer	4 bytes	NOT NULL, Foreign Key	An inpatient must be taken care of by one nurse whenever admitted.
fee	integer	4 bytes	NOT NULL fee >= 0	Fee cannot be null and must be positive.
Primary key	(pid, date_of_adm)			

Foreign key behavior:

pid

- On update cascade: Update as pid of current patient is updated.
- On delete cascade: Delete if patient is deleted from the database.

nurse_id

- On update cascade: Update as ecode of current nurse is updated.
- On delete restrict: If a nurse is taking care of any inpatient, that nurse cannot be deleted.

Treat				
Column	Data type	Data length	Constraints	Explanation
pid	integer	4 bytes	NOT NULL, Foreign Key	Unique identifier for each treatment.
date_of_adm	date	4 bytes	NOT NULL	Whenever patient is admitted, the date must be recorded.
did	integer	4 bytes	NOT NULL, Foreign Key	Can be NULL since the patient has not been discharged.
sid	integer	4 bytes	NOT NULL, Foreign Key	
Primary key	(did,sid)			

Foreign key behavior:

pid, date_of_adm:

- On update cascade: update as primary key of admission is updated
- On delete cascade: if the admission is deleted, all relating treat information should be deleted.

did:

- On update cascade: update as primary key of admission is updated
- On delete restrict: if there is still treatment, admission, treat relating to a doctor, that doctor must not be deleted from the database (for recognition and reponsibility).

sid:

- On update cascade: update as primary key of treatment is updated
- On delete cascade: if the treatment is deleted, all relating treat information should be deleted.

T_Consists_Of / E_consists_of				
Column	Data type	Data length	Constraints	Explanation
sid	integer	4 bytes	NOT NULL, Foreign Key	Unique identifier for each treatment.
mid	integer	4 bytes	NOT NULL, Foreign Key	Whenever patient is admitted, the date must be recorded.
quantity	integer	4 bytes	NOT NULL Positive	Data type integer can hold large number in case the quantity of medication in batch is large.
Primary key	(sid,mid)			

Foreign key behavior:

sid:

- On update cascade: Update as sid of healthcare service is updated.
- On delete cascade: If healthcare service is deleted, all associated information is deleted.

mid:

- On update cascade: Update as mid of medication is updated.
- On delete restrict: Medication can not be deleted from database if some healthcare service contains information about that medication.

Examination				
Column	Data type	Data length	Constraints	Explanation
sid	integer	4 bytes	NOT NULL, Primary Key	Unique identifier for each treatment.
fee	integer	4 bytes	NOT NULL, Positive or 0	The INTEGER type is sufficient for storing monetary values. The NOT NULL and CHECK constraints ensure the fee is valid and non-negative.
e_date	date	4 bytes	NOT NULL	NOT NULL since e_date must be recorded when an examination is recorded. Also, examination is also important in terms of patient history.
diagnosis	varchar(200)	At most 200 bytes		NULLABLE because there may be no diagnosis, e.g the patient is completely fine.
next_exam_date	date	4 bytes	NULL OR next_exam_date >= e_date)	Enforces logical sequencing of examination dates
pid	integer	4 bytes	NOT NULL, Foreign Key	
did	integer	4 bytes	NOT NULL, Foreign Key	
med_f	boolean	1 byte	NOT NULL Default FALSE	If not specified, assume that there is no medication associated with healthcare service.

Foreign key behavior:

pid:

- On update cascade: Update as pid of current outpatient is updated.
- On delete cascade: If patient is deleted, all relating information is deleted.

did:

- On update cascade: Update as pid of current outpatient is updated.
- On delete set null: If the doctor is deleted, set null since lacking doctor information for examination may not be too severe. Patient history is rather more important.

1.3 Triggers

Triggers automate certain actions within the database.

1.3.1 Department

- `validate_dept_mgr()`
    * Time: before insert or update
    * Use case: Ensures that the dean of the department must be an employee working in that department.

1.3.2 Inpatient/Outpatient

- `insert_patient_ipid` or `insert_patient_opid`
    * Time: before insert.
    * Use case: Automatically generate OPID, IPID with format 'OPXXXXXXXXX','IPXXXXXXXXX' (X is digit) respectively whenever a new patient is added to the corresponding table.
- `pid_consistency_outpatient` or `pid_consistency_inpatient`
    * Time: before insert.
    * Use case: Ensure information of the same patient (same pid) in two tables (inpatient and outpatient) are consistent whenever add a patient who already occurred in the other table (either inpatient or outpatient). If not consistent, raise error.
- `patient_consistency_update_outpatient` or `patient_consistency_update_inpatient`
    * Time: after update.
    * Use case: If a patient exists in both inpatient and outpatient tables, whenever information of that patient updated in one table the system automatically update that in the other table.
- `tg_validate_phone_inpatient` or `tg_validate_phone_outpatient`
    * Time: before insert or update.
    * Use case: Ensure that phone number of a patient is in valid format (Vietnam context).

1.3.3 Doctor/Nurse

- `tg_validate_phone_number_doctor` or `tg_validate_phone_number_nurse`
    * Time: before insert or update.
    * Use case: Ensure that phone numbers of a employee is in valid format (Vietnam context).

1.4 Functions

All functions that are used are add-ups to support triggers will not be mentioned in this part.

- `insert_patient(pid,target_table)`
    * Parameter: pid: patient's pid, target_table: table to be inserted.
    * Use case: Use this function to insert a patient who already existed in either inpatient or outpatient table with ease. This function will perform copy action for all attributes except for OPID or IPID.
- `update_exp_flags_periodically()`
    * Parameter: None
    * Use case: This function will be executed periodically (extension pg_cron) to automatically marked the medication that are expired.

2 Stored Procedures and Functions

2.1 Question a

Increase Inpatient Fee to 10% for all the current inpatients who are admitted to hospital from 01/09/2020.

SQL

```
UPDATE admission
SET fee = fee * 1.1
WHERE date_of_disc IS NULL
AND date_of_adm >= TO_DATE('2020-09-01', 'yyyy-mm-dd');
```

Result

Data after running the function:

pid	date_of_adm	date_of_disc	room	nurse_id	diagnosis	fee
94	2022-12-18		110	40	Alzheimer disease	1100000
148	2023-05-03		301	19	Cardiac arrest	990000
62	2023-07-16		318	19	Stroke: ischemic or hemorrhagic	1210000
67	2023-09-12		312	30	Placenta previa	770000
68	2022-11-04		311	28	Postpartum hemorrhage	2420000
69	2023-11-05		104	38	Alzheimer disease	550000
73	2023-01-12		ICU-213	32	Shock: septic, cardiogenic, anaphylactic	3190000
76	2023-06-06		110	18	Trauma: fractures, head injuries, soft tissue injuries	550000
96	2022-03-04		SR-116	22	Fractures: surgical intervention required	880000
187	2022-08-10		208	20	Respiratory distress: acute asthma, pneumonia	330000
60	2022-09-14		209	27	Placenta previa	440000
79	2022-02-25		SR-210	24	Appendicitis	2420000
84	2023-07-18	2023-11-15	301	18	Poisoning: overdose, toxic ingestion	200000
85	2023-09-28	2024-05-08	SR-218	21	Peritonitis: intra-abdominal infection	1900000
86	2023-04-19	2024-03-18	318	29	Normal pregnancy and delivery	500000
87	2022-03-23	2024-02-07	ICU-107	34	Sepsis	2800001
88	2023-03-28	2024-10-07	217	28	Postpartum hemorrhage	600001
89	2022-05-21	2024-03-27	ICU-211	34	Traumatic brain injury: severe	700000
90	2022-07-27	2023-05-16	211	28	Placenta previa	1100000
91	2023-05-31	2024-07-09	215	19	Trauma: fractures, head injuries, soft tissue injuries	1600000

Data before running the function:

pid	date_of_adm	date_of_disc	room	nurse_id	diagnosis	fee
94	2022-12-18		110	40	Alzheimer disease	1100000
148	2023-05-03		301	19	Cardiac arrest	990000
62	2023-07-16		318	19	Stroke: ischemic or hemorrhagic	1210000
67	2023-09-12		312	30	Placenta previa	770000
68	2022-11-04		311	28	Postpartum hemorrhage	2420000
69	2023-11-05		104	38	Alzheimer disease	550000
73	2023-01-12		ICU-213	32	Shock: septic, cardiogenic, anaphylactic	3190000
76	2023-06-06		110	18	Trauma: fractures, head injuries, soft tissue injuries	550000
96	2022-03-04		SR-116	22	Fractures: surgical intervention required	880000
187	2022-08-10		208	20	Respiratory distress: acute asthma, pneumonia	330000
60	2022-09-14		209	27	Placenta previa	440000
79	2022-02-25		SR-210	24	Appendicitis	2420000
84	2023-07-18	2023-11-15	301	18	Poisoning: overdose, toxic ingestion	200000
85	2023-09-28	2024-05-08	SR-218	21	Peritonitis: intra-abdominal infection	1900000
86	2023-04-19	2024-03-18	318	29	Normal pregnancy and delivery	500000
87	2022-03-23	2024-02-07	ICU-107	34	Sepsis	2800001
88	2023-03-28	2024-10-07	217	28	Postpartum hemorrhage	600001
89	2022-05-21	2024-03-27	ICU-211	34	Traumatic brain injury: severe	700000
90	2022-07-27	2023-05-16	211	28	Placenta previa	1100000
91	2023-05-31	2024-07-09	215	19	Trauma: fractures, head injuries, soft tissue injuries	1600000

The fee value of the patient who have date_of_disc (date of discover) is a null value and date_of_adm (date of admission) is after 2020-09-01 will be increase by 10% because the null value of date_of_disc means this patient have not recover yet.

2.2 Question b

Select all the patients (outpatient & inpatient) of the doctor named 'Nguyen Van A'.

We do not have doctor's name 'Nguyen Van A' so we randomly select the name of one doctor that we have: 'Nguyen Anh'.

SQL

```sql
select distinct o.pid, lname, fname, dob, address, phone, gender
from outpatient o
join examination e
on o.pid = e.pid
where did = (select ecode from doctor d where concat(lname,' ',fname) = 'Nguyen Anh')
union
select distinct i.pid, lname, fname, dob, address, phone, gender
from inpatient i
join treat t
on i.pid = t.pid
where did = (select ecode from doctor d where concat(lname,' ',fname) = 'Nguyen Anh')
    ;
```

Result is a list of patient that doctor Nguyen Anh treat or examine:

Note: a patient can be duplicate because this patient is an outpatient and also inpatient too (EX: patient with pid = 127).

```
104|OP000000104|Do    |Phat  |1963-04-02|127 Nguyen Van Quy, District 7, Ho Chi Minh City           |0954947838|M     |
106|OP000000106|Vu    |Cam   |2009-11-21|26 Ton Duc Thang, Dong Da District, Hanoi                  |0973359432|F     |
108|OP000000108|Ly    |Hoa   |1935-08-31|105 Huynh Tinh Cua, District 3, Ho Chi Minh City           |0988367835|M     |
110|OP000000110|Vu    |Tuyet |1987-06-04|125 Nguyen Chi Thanh, Hai Chau District, Da Nang           |0928728842|F     |
111|OP000000111|Pham  |Anh   |2007-02-03|105 Nguyen Tri Phuong, Thanh Khe District, Da Nang         |0921672211|F     |
118|OP000000118|Ly    |Giang |1981-09-02|79 Quang Duc, Huong Long District, Hue                     |0377375428|M     |
119|OP000000119|Bui   |Quan  |2018-07-31|26 Phu Xuan, Kim Long District, Hue                        |0916661156|M     |
120|OP000000120|Duong |Hoa   |1989-05-20|82 Tran Nao, Thu Duc City, Ho Chi Minh City                |0344241847|F     |
122|OP000000122|Duong |Tuan  |2005-05-20|72 Tran Khat Chan, Hai Ba Trung District, Hanoi            |0911747919|M     |
123|OP000000123|Nguyen|Tung  |2000-11-20|105 Le Do, Thanh Khe District, Da Nang                     |0961152543|M     |
124|OP000000124|Ho    |Chinh |2015-08-22|39 Nguyen Hue, O Mon District, Can Tho                     |0964832766|M     |
126|OP000000126|Phan  |Dai   |1999-05-18|125 Kham Thien, Dong Da District, Hanoi                    |0965534276|M     |
127|IP000000127|Ly    |Lam   |1961-01-25|38 Pham Van Chieu, District 12, Ho Chi Minh City           |0981329763|M     |
127|OP000000127|Ly    |Lam   |1961-01-25|38 Pham Van Chieu, District 12, Ho Chi Minh City           |0981329763|M     |
131|OP000000131|Hoang |Cam   |2018-10-27|181 Le Van Chi, Thu Duc City, Ho Chi Minh City             |0976546573|F     |
135|OP000000135|Tran  |Hoa   |1987-04-26|87 Thuy Khue, Tay Ho District, Hanoi                       |0361191769|F     |
137|OP000000137|Hoang |Anh   |1990-10-13|34 Dong Da, Phu Hau District, Hue                          |0955892298|F     |
141|OP000000141|Nguyen|Son   |1964-12-07|145 Nguyen Son, Long Bien District, Hanoi                  |0932324276|M     |
143|OP000000143|Duong |Chau  |1973-06-21|127 Tran Hung Dao B, District 5, Ho Chi Minh City          |0972943136|F     |
145|OP000000145|Phan  |Dai   |1953-09-17|30 Hoa An, Huong Long District, Hue                        |0968485568|M     |
146|OP000000146|Ngo   |Khoa  |1943-04-08|195 Truong Sa, Ngu Hanh Son District, Da Nang              |0395384479|M     |
154|OP000000154|Phan  |Van   |1979-02-07|103 Nguyen Van Tan, Cam Le District, Da Nang               |0932167942|M     |
156|OP000000156|Nguyen|Yen   |1994-09-04|56 Nguyen Dinh Chinh, District Phu Nhuan, Ho Chi Minh City |0393971621|F     |
163|OP000000163|Dang  |Anh   |1973-11-23|111 Bach Dang, Hai Chau District, Da Nang                  |0391937661|F     |
187|OP000000187|Dang  |Khanh |1948-06-28|186 Thanh Xuan Bac, Ha Dong District, Hanoi                |0395564486|F     |
188|OP000000188|Phan  |Thuy  |1931-09-21|55 Nguyen Van Thoai, Son Tra District, Da Nang             |0979268294|F     |
191|OP000000191|Tran  |Chinh |1992-07-21|46 Le Trong Tan, Cam Le District, Da Nang                  |0911721674|M     |
198|OP000000198|Vu    |Thi   |2011-09-22|57 Hang Gai, Hoan Kiem District, Hanoi                     |0985775464|F     |
200|OP000000200|Pham  |Ly    |1954-07-26|10 Nguyen Van Luong, District 12, Ho Chi Minh City         |0911271867|F     |
201|OP000000201|Duong |Dung  |1974-04-01|67 To Ky, Cam Le District, Da Nang                         |0924239849|M     |
202|OP000000202|Bui   |Nhung |1963-07-07|150 Nguyen Trong Tuyen, District Phu Nhuan, Ho Chi Minh City|0926897754|F     |
205|IP000000205|Ngo   |Thi   |1943-10-28|125 Tran Binh Trong, District 5, Ho Chi Minh City          |0961519392|F     |
208|OP000000208|Tran  |Thuy  |1928-09-29|62 Dang Van Bi, Thu Duc City, Ho Chi Minh City             |0314731893|F     |
217|OP000000217|Hoang |Dai   |2013-07-27|176 Dong Da, Hai Chau District, Da Nang                    |0355281593|M     |
230|OP000000230|Hoang |Hang  |1990-08-12|175 Dong Ba, Vy Da District, Hue                           |0953411398|F     |
```

2.3 Question c

Write a function to calculate the total medication price a patient has to pay for each treatment or examination.

SQL

```sql
CREATE OR REPLACE FUNCTION get_medication_costs_by_patient(patient_id INT)
RETURNS TABLE (
    sid INT,
    total_cost BIGINT  -- Adjust type as needed
) AS $$
BEGIN
    RETURN QUERY
    select distinct x.sid, x.total_cost
    from (
        (select distinct tco.sid, sum(quantity * price) as total_cost
        from t_consists_of tco
        join medication m
        on m.medication_id = tco.mid
        join treat t
        on tco.sid = t.sid
        where t.pid = patient_id
```

```
        group by tco.sid)
    union
        (select distinct eco.sid, sum(quantity * price) as total_cost
        from e_consists_of eco
        join medication m
        on m.medication_id = eco.mid
        join examination e
        on eco.sid = e.sid
        where e.pid = patient_id
        group by eco.sid)
        ) x
    order by x.sid asc;
END;
$$ LANGUAGE plpgsql;
```

Run the function by the following query:

```
SELECT * FROM get_medication_costs_by_patient(102);
```

Result is a list of service (sid) and the total medication cost for that service of the given patient (pid is user input):

sid	total_cost
1	1600000
2	615000
5	960000
277	960000
340	60000
366	1160000

2.4 Question d

Write a procedure to sort the doctor in increasing number of patients he/she takes care in a period of time

SQL

```
CREATE OR REPLACE PROCEDURE get_patient_counts(
    start_date DATE,
    end_date DATE
)
```

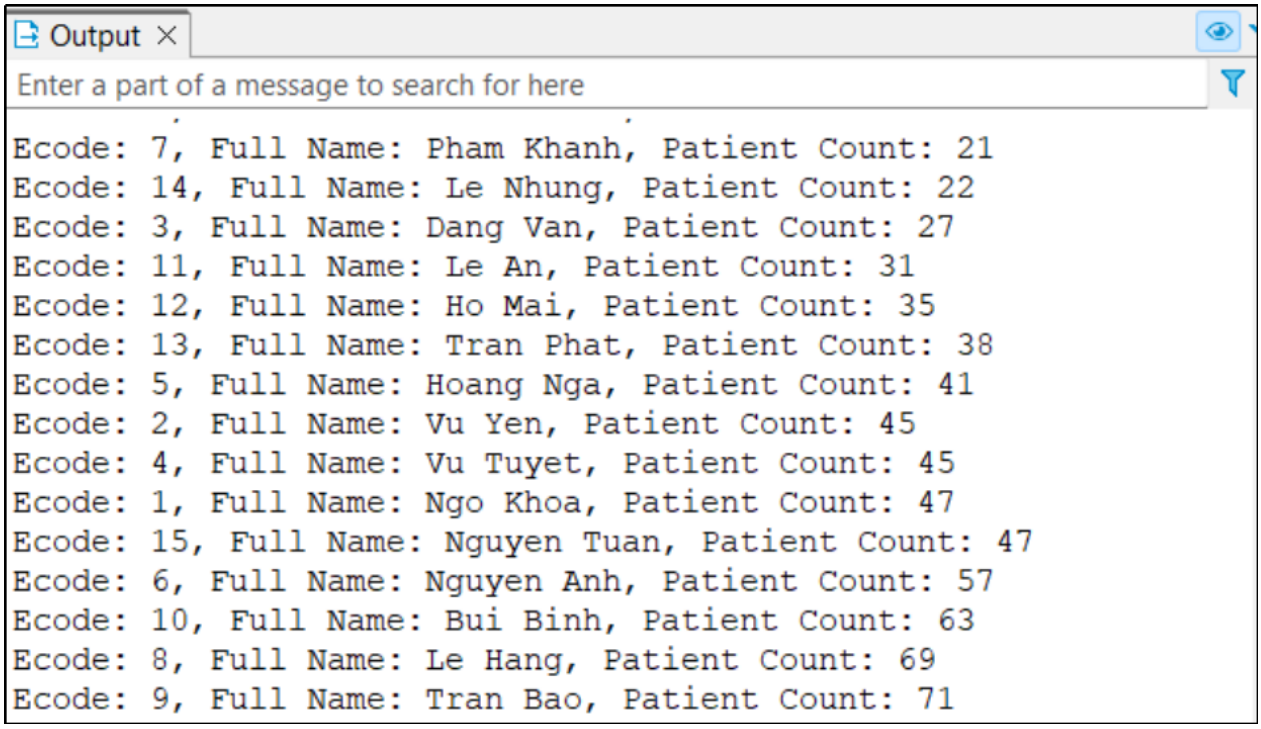
```
LANGUAGE plpgsql
AS $$
DECLARE
    record RECORD;
BEGIN
    FOR record IN
        SELECT DISTINCT d.ecode,
                        CONCAT(d.lname, ' ', d.fname) AS full_name,
                        COUNT(DISTINCT x.pid) AS patient_count
        FROM doctor d
        JOIN (
            (SELECT t.pid, t.did
             FROM treat t
             JOIN admission a ON a.pid = t.pid
             WHERE (a.date_of_disc > start_date AND a.date_of_adm <= end_date) OR
                   (a.date_of_adm < end_date AND a.date_of_disc IS NULL))
            UNION
            (SELECT e.pid, e.did
             FROM examination e
             WHERE e.e_date >= start_date AND e.e_date <= end_date)
        ) x ON d.ecode = x.did
        GROUP BY d.ecode, d.lname, d.fname
        ORDER BY COUNT(DISTINCT x.pid) ASC
    LOOP
        RAISE NOTICE 'Ecode: %, Full Name: %, Patient Count: %', record.ecode, record
    .full_name, record.patient_count;
    END LOOP;
END;
$$;
```

Run the procedure by the following query:

```
CALL get_patient_counts('2000-09-28', '2024-12-02');
```

Result is a list of doctor and the number of patient (include inpatient and outpatient) that doctor treat or examine and sort ascendent by the number of count in a period of time (user input):

Note: Total number of patient_count is higher than total number of patient because one doctor can treat or examine many patient at a time.



3 Building Application

This section describes the technology we use to build the application. This application will connect to the database created in Part 1 and Part 2. It will allow users to view data and interact with the database via a website interface.

3.1 Programming Technology

The application is built using a modern and efficient technology stack, which includes:

- **Frontend:** React TS for building user interfaces.
- **Backend:** FastAPI (Python web framework) to handle API requests and connect with PostgreSQL.
- **Database Management System:** pgAdmin (PostgreSQL)
- **Version Control:** Git for collaborative development.

3.2 Create User

```
1 CREATE ROLE Manager WITH LOGIN PASSWORD 'anhinla';
2 -- Granting privileges
3 GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO Manager;
4 GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO Manager;
5 GRANT ALL PRIVILEGES ON ALL FUNCTIONS IN SCHEMA public TO Manager;
6 GRANT EXECUTE ON ALL PROCEDURES IN SCHEMA public TO Manager;
```

```
GRANT TRIGGER ON ALL TABLES IN SCHEMA public TO Manager;
GRANT CREATE ON SCHEMA public TO Manager;
GRANT USAGE ON SCHEMA public TO Manager;
```

When a manager of our hospital log in to the application, he/she will access the database with role Manager. The access of Manager will be limited to the `public` schema and the specific privileges granted on tables, sequences, functions,... Manager can grant privileges within the scope of the `public` schema or objects they have access to. Only the DBA of hospital database has full control over the entire PostgreSQL instance, bypassing all access controls and security mechanisms, and can manage the server and perform administrative tasks. We grants privileges to Manage while considering the Principle of Least Privilege and the separation of duties.

3.3 Functionality

Key functionalities of the application include:

3.3.1 Authentication

We restrict access to our application exclusively to Manager accounts. Authentication is based on email and a strong password designed to prevent brute-force attacks. Additionally, we issue an access token and a refresh token, which are required to be included in the request headers. These tokens ensure that only authorized Managers can send requests to our backend server.

3.3.2 Add new patient

We offer two options for adding a patient: a completely new patient or a returning (visited) patient. For returning patients, we provide the option to update their personal information, such as phone number, address, or to change their status between outpatient and inpatient. To ensure data consistency across the application, we use triggers and functions within our database. When a patient's information is updated, the changes are reflected in both the Inpatient and Outpatient tables. Additionally, we allow the manager to add an examination for outpatient cases and an admission for inpatient cases.

3.3.3 Get specific patient information (personal info + medical records)

We enable Managers to search for patients using various criteria, including patient ID, first name, last name, or full name. Each patient has a dedicated page that displays their personal examination details along with their medical history, which includes treatments and examinations. For each treatment or examination, specific information is provided, such as the names of the doctors or nurses involved, as well as details regarding medication payments, including medication name, price, and total fees.

3.3.4 Get specific doctor information (personal info + work history)

For doctors, we implement a similar mechanism as with patients. However, instead of viewing medical history, each doctor's page displays their personal information and work history. This includes details of the treatments they have participated in and the patients associated with each specific treatment.

4 Database Management

This section highlights database management techniques, focusing on indexing and security.

4.1 Indexing Efficiency

In our hospital database design, an inpatient can have multiple admissions, each admission can involve multiple treatments, and each treatment can involve multiple doctors. As a result, the Treat and Treatment tables are expected to store a substantial volume of data. To enhance query performance and ensure efficient data retrieval, we utilize indexing on these tables.

In our case, we want to retrieve information of all treatments of a specific patient after a specific date. The example SQL query:

```
1 select inpatient.pid,inpatient.lname, inpatient.fname, inpatient.phone, admission.
    date_of_adm, admission.room, admission.nurse_id, treat.did, treatment.start_date,
     treatment.end_date, treatment.result
2 from inpatient
3 join admission on inpatient.pid = admission.pid
4 join treat on  admission.pid = treat.pid and admission.date_of_adm = treat.
    date_of_adm
5 join treatment on treat.sid = treatment.sid
6 where treat.date_of_adm > '2023-01-01' and treat.pid = 456509;
```

In our DBMS, `Inpatient` table has 111076 rows, `Admission` table has 111076 rows, `Treat` table has 766969 rows and `Treatment` table has 255640 rows.Given this significant amount of data, performing a query without an index would require a sequential scan, where every row in the Treat table is checked. This process is computationally expensive and involves a large number of disk reads.

With the indexing, the database can jump directly to relevant rows, reducing the number of disk reads. The database can leverage the index not only for filtering but also for joining operations, resulting in faster execution and lower computational overhead.

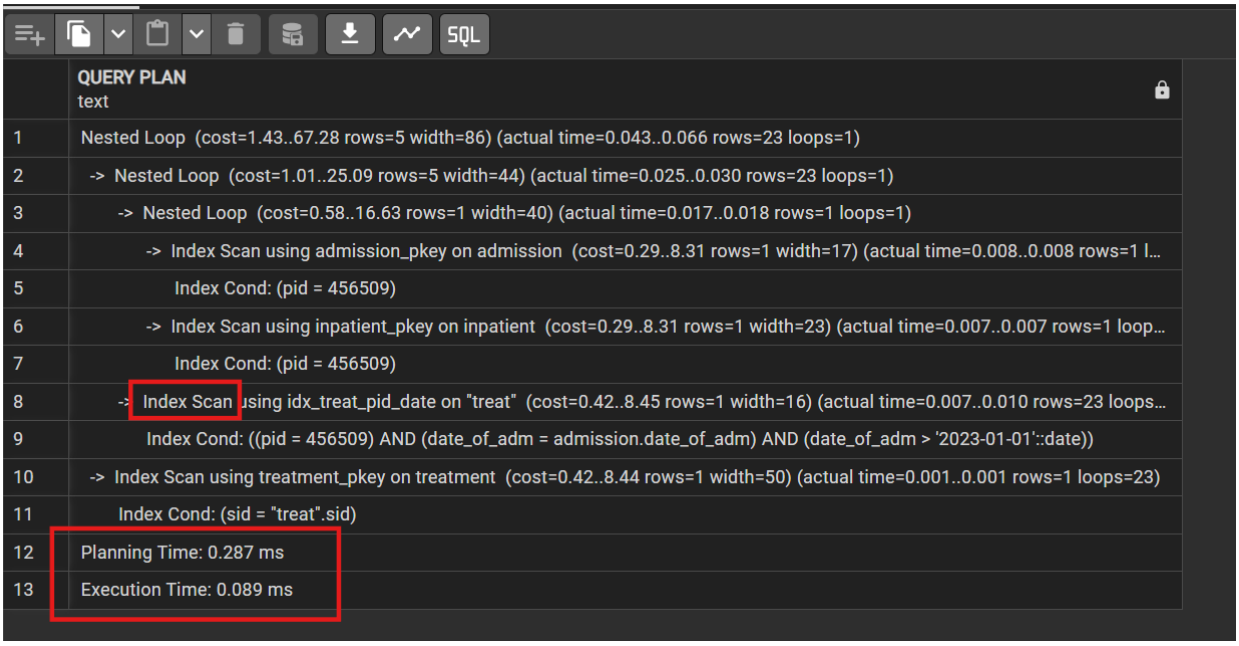
Because both date_of_adm and pid columns of `Treat` table are involved in filtering and joining so we choose to use B+ tree indexing to optimize query performance.

```sql
CREATE INDEX idx_treat_pid_date ON treat(pid, date_of_adm);
```

Performance Comparison

	QUERY PLAN text
1	Nested Loop (cost=15650.60..28483.87 rows=5 width=86) (actual time=87.101..87.975 rows=23 loops=1)
2	Join Filter: (admission.date_of_adm = "treat".date_of_adm)
3	-> Nested Loop (cost=0.00..5544.91 rows=1 width=40) (actual time=8.233..8.469 rows=1 loops=1)
4	-> Seq Scan on inpatient (cost=0.00..3019.45 rows=1 width=23) (actual time=4.352..4.458 rows=1 loops=1)
5	Filter: (pid = 456509)
6	Rows Removed by Filter: 111075
7	-> Seq Scan on admission (cost=0.00..2525.45 rows=1 width=17) (actual time=3.876..4.005 rows=1 loops=1)
8	Filter: (pid = 456509)
9	Rows Removed by Filter: 111075
10	-> Hash Join (cost=15650.60..22938.90 rows=5 width=58) (actual time=78.860..79.491 rows=23 loops=1)
11	Hash Cond: (treatment.sid = "treat".sid)
12	-> Seq Scan on treatment (cost=0.00..5051.40 rows=255640 width=50) (actual time=0.015..20.257 rows=255640 loop...
13	-> Hash (cost=15650.53..15650.53 rows=5 width=16) (actual time=32.848..32.849 rows=23 loops=1)
14	Buckets: 1024 Batches: 1 Memory Usage: 10kB
15	-> Seq Scan on "treat" (cost=0.00..15650.53 rows=5 width=16) (actual time=32.462..32.825 rows=23 loops=1)
16	Filter: ((date_of_adm > '2023-01-01'::date) AND (pid = 456509))
17	Rows Removed by Filter: 766946
18	Planning Time: 0.550 ms
19	Execution Time: 88.045 ms

Figure 4.1.1: Analysis report when running the above SQL query using sequential scan



	QUERY PLAN text
1	Nested Loop (cost=1.43..67.28 rows=5 width=86) (actual time=0.043..0.066 rows=23 loops=1)
2	-> Nested Loop (cost=1.01..25.09 rows=5 width=44) (actual time=0.025..0.030 rows=23 loops=1)
3	-> Nested Loop (cost=0.58..16.63 rows=1 width=40) (actual time=0.017..0.018 rows=1 loops=1)
4	-> Index Scan using admission_pkey on admission (cost=0.29..8.31 rows=1 width=17) (actual time=0.008..0.008 rows=1 l...
5	Index Cond: (pid = 456509)
6	-> Index Scan using inpatient_pkey on inpatient (cost=0.29..8.31 rows=1 width=23) (actual time=0.007..0.007 rows=1 loop...
7	Index Cond: (pid = 456509)
8	-> Index Scan using idx_treat_pid_date on "treat" (cost=0.42..8.45 rows=1 width=16) (actual time=0.007..0.010 rows=23 loops...
9	Index Cond: ((pid = 456509) AND (date_of_adm = admission.date_of_adm) AND (date_of_adm > '2023-01-01'::date))
10	-> Index Scan using treatment_pkey on treatment (cost=0.42..8.44 rows=1 width=50) (actual time=0.001..0.001 rows=1 loops=23)
11	Index Cond: (sid = "treat".sid)
12	Planning Time: 0.287 ms
13	Execution Time: 0.089 ms

Figure 4.1.2: Analysis report when running the above SQL query using index scan

Planning time: time the database takes to parse, analyze, and optimize the query before execution.

- Without indexing: **0.550 ms** (higher because it has to analyze more potential query paths without indexes).
- Indexing: **0.287 ms** (lower because indexes simplify query optimization by narrowing down query paths).

Execution time: actual time taken to execute the query and retrieve results from the database.

- Without indexing (sequential scan): **88.045 ms** (significantly slower due to scanning all rows sequentially in the table).
- Indexing (Index scan): **0.089 ms** (due to the index providing direct access to the relevant rows, avoiding the need to scan the entire table).

Cost estimates:

- Without indexing (sequential scan): The cost is 15650.60..28483.87 (rows=5)
- Indexing (Index scan): The cost is 1.43..67.28 (rows=5). The Index Scan has a much lower cost estimate, suggesting that it will be much more efficient in terms of resource usage.

Rows Processed: Both plans process 5 rows in the query, but the Index Scan does this much faster and with less computational overhead.

In our case, the **Index scan** is more effective than **Sequential Scan**, with significantly faster planning and execution times, leading to reduced resource consumption. This indicates the use of indexing when dealing

with large datasets to optimize database performance. However, it's important to note that overusing indexes can lead to some drawbacks, such as increased storage requirements and slower write operations (INSERT, UPDATE, DELETE), as the database must also update the indexes.

4.2 Database Security

One use-case of database security in our application is avoiding **SQL injection attack**. As mentioned before, we use FastAPI (Python) to handle our back-end services like authentication or working with the database. I use Parameterized Statements (Bind variables) along with SQL Alchemy's ORM (Object Relational Mapping) to secure my website from SQL injection attack.

Attackers may exploit our program in the following ways if it lacks a SQL injection prevention mechanism:

4.2.1 Risks

Using SQL injection to Authenticate as Administrator:

Consider a simple authentication system using a database table with usernames and passwords:

```
sql= "SELECT id FROM users WHERE username='" + user + "' AND password='" + pass + "'"
```

The attacker can provide the string *pass*: `password' OR 5=5`.

```
SELECT id FROM users WHERE username='user' AND password='pass' OR 5=5
```

Because $5 = 5$ is always true, the entire WHERE statement will be true, regardless of the username or password provided. As a consequence, attacker can bypass the authentication process.

Using SQL injection to access sensitive data: If our application has code that corresponding to the below SQL query:

```
SELECT * FROM items
WHERE owner =
AND itemname = ;
```

The attacker can provide the string *itemname*: `Widget' OR 5=5`. Which is the same as: `SELECT * FROM items;`

This means the query will return the data of the entire table, giving the attacker unauthorized access to sensitive data.

Injecting Malicious Statements into Form Field

If attacker inputs *firstname*: `malicious'ex`

```
SELECT id, firstname, lastname FROM authors WHERE firstname = 'malicious'ex' and
    lastname ='newman'
```

The database identifies incorrect syntax due to the single apostrophe, and our back end server will be crashed.

4.2.2 Our solution

Authentication: We have only one Manager account. The valid email and password are stored securely in environment variables (.env file). This means that there are no SQL queries based on user input in the authentication logic, completely eliminating the possibility of SQL injection. Passwords are not stored as plain text but are hashed using a secure hashing algorithm. Even if an attacker gains access to the .env file, the hashed password cannot be easily reversed to its original value. Upon successful authentication, access and refresh tokens are generated for further interactions with the system.

Sensitive data attack and syntax error-based attack:

```
query = select(Doctor)
        .join(Department,Department.dcode == Doctor.dcode)
        .where(func.lower(Department.dtitle) == dtitle.lower())
```

SQLAlchemy automatically binds parameters to the query, ensuring that any user-provided input is treated as data, not executable SQL code. By using SQLAlchemy's Object Relational Mapping (ORM), raw SQL queries are avoided. Instead, the query is built programmatically using Python objects and methods, which SQLAlchemy translates into secure SQL statements. The ORM layer ensures that the final SQL query is syntactically valid and escapes any malicious characters in the input. SQLAlchemy automatically escapes special characters such as single quotes (') and double dashes (–) that attackers often use to inject malicious SQL statements. For example, if a user provides input like ecode = 123' OR 1=1 –, it will not alter the SQL logic because the input is treated as a literal string or integer, not executable code.

```
class ExaminationAddModel(BaseModel):
    fee: int
    diagnosis: Optional[str]
    next_exam_date: Optional[date] = None
    pid: int
    did: int
async def add_examination(
    examination_data:ExaminationAddModel,
    session:AsyncSession=Depends(get_session),
    user_details=Depends(access_token_bearer)):
```

Furthermore, we use strictly-typed request body validation on the incoming request body. Each field is validated to ensure it adheres to the defined data type. Our models are derived from - Pydantic's BaseModel. Pydantic ensures that inputs are parsed into their expected types:

- Strings are sanitized and properly escaped.

- Dates are converted into Python date objects, ensuring invalid date formats.

Integers are parsed strictly, rejecting non-numeric values.

If an attacker submits a request body with malicious data, invalid types, or missing mandatory fields, malicious inputs will be caught at the request validation stage before any database interaction occurs. This strict validation prevents attackers from exploiting vulnerabilities, as their malicious inputs are rejected outright, blocking any further actions and safeguarding the database from unauthorized access or manipulation.

5 Link

Diagrams: Google Drive Link

Data + Script: Google Drive Link

Application source code repository: hospital-app